GitLab Branching Strategy

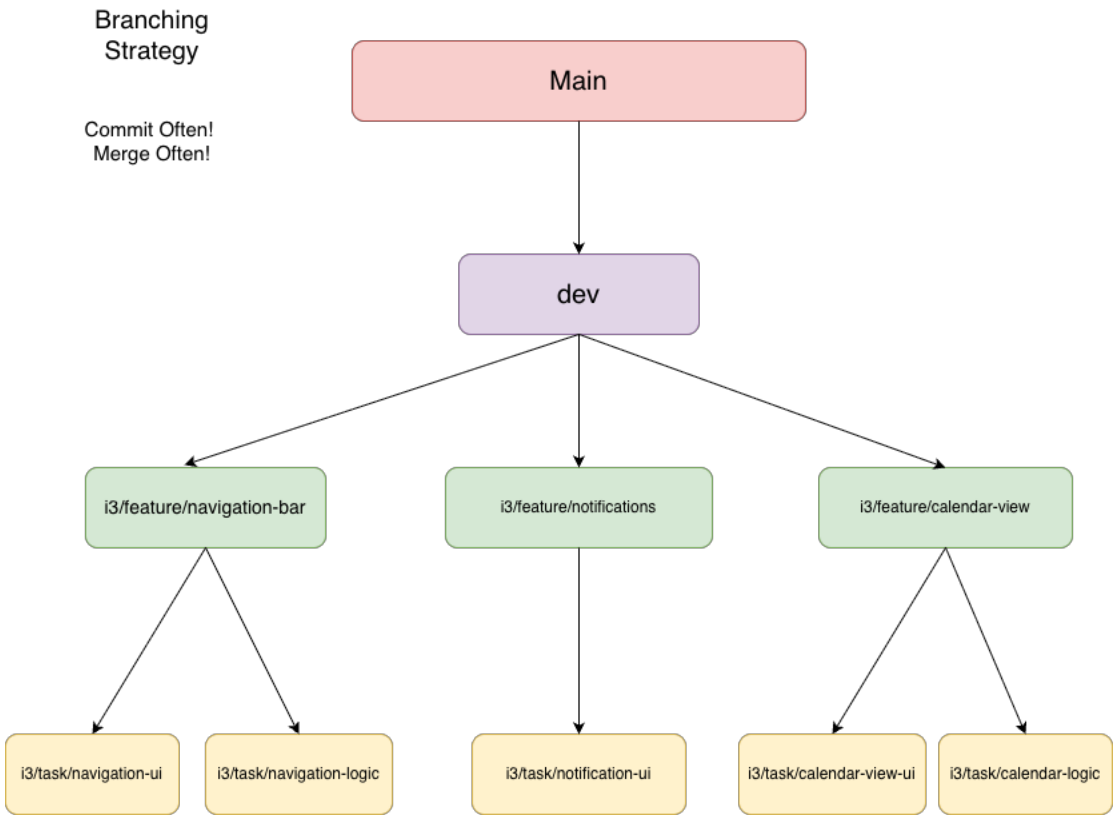
Git Girls – i3

1. Overview

This document defines the GitLab branching strategy for the Android group project. All team members must follow this structure to ensure clean integration, traceability, and smooth collaboration throughout the development lifecycle.

The branching model follows a three-tier hierarchy:

- main — production-ready, stable code only
- dev — active integration branch for ongoing development
- i3/feature/short-description — isolated feature development
- i3/task/short-description — sub-tasks within each feature



2. Branch Hierarchy

2.1 main

The main branch represents the stable, deployable state of the application. It is protected and should only receive merges from dev after thorough testing and team approval.

- Never commit directly to main
- All merges into main require a Merge Request (MR) with at least one approval
- Tags/releases are created from main

2.2 dev

The dev branch is the central integration branch. All completed feature branches are merged here. This branch should remain buildable at all times.

- Feature branches are created from dev
- Feature branches merge back into dev via Merge Requests
- dev is merged into main at sprint milestones after QA sign-off

2.3 Feature Branches

Feature branches are created directly from dev and represent a self-contained piece of functionality. They follow the naming convention below.

Format	`i3/feature/short-description`

Examples:

```
i3/feature/cycle-tracking
i3/feature/symptom-logging
i3/feature/period-predictions
```

- Use lowercase and hyphens only — no spaces or underscores
- Keep descriptions short but meaningful (2–4 words)
- One feature branch per logical feature, not per developer

2.4 Task Branches

Task branches are created from their parent feature branch and represent a single, granular unit of work. When a task is complete it is merged back into its feature branch.

Format	`i3/task/short-description`

Examples:

```
i3/task/cycle-start-date-input
i3/task/flow-intensity-selector
i3/task/next-period-date-calc
```

- Always branch from the correct parent feature branch
- Keep each task branch focused on a single concern
- Once merged, task branches may be deleted to keep the repo clean

3. Day-to-Day Workflow

Step	Action	Command / Notes
1	Update dev locally	`git checkout dev` `git pull origin dev`
2	Create a feature branch	`git checkout -b i3/feature/my-feature`
3	Create a task branch	`git checkout -b i3/task/my-task`
4	Work on your task	Commit regularly with clear commit messages
5	Push task branch	`git push origin i3/task/my-task`
6	Open MR: task → feature	Merge into the parent feature branch on GitLab
7	Open MR: feature → dev	Once all tasks are complete; require peer review
8	Open MR: dev → main	At sprint end; requires team lead approval

4. Merge Request Rules

All merges must go through GitLab Merge Requests (MRs) — no direct pushes to protected branches.

MR Type	Reviewer Required	Approvals Needed
task → feature	Any team member	1

feature → dev	Another developer (not author)	1
dev → main	Team lead / all members	2+

5. Commit Message Guidelines

Use clear, imperative commit messages that describe what the commit does, not what you did.

Format	`[type]: short description (50 chars max)`

Types:

- feat — a new feature or UI element
- fix — a bug fix
- refactor — code restructure with no behaviour change
- test — adding or updating tests
- docs — documentation only changes
- chore — build tasks, dependencies, config

Good examples:

```
feat: add login button to home screen
fix: resolve crash on null user token
refactor: extract auth logic into service class
```

6. Quick Reference Card

✓ DO	✗ DON'T
Always pull dev before creating a branch	Push directly to main or dev
Use the correct naming convention	Use spaces or capitals in branch names
Open an MR for every merge	Merge without a code review
Write clear commit messages	Commit unrelated changes together
Delete task branches after merge	Leave stale branches open indefinitely
Keep feature branches short-lived	Let branches diverge too far from dev